Exp No: 1	**Create a simple webpage using HTML and CSS**
Date:	

Aim

To create a simple webpage with structured content and styling using HTML and CSS.

Algorithm

1. Start
2. Create an HTML file (index.html).
3. Define the basic HTML structure including <!DOCTYPE html>, <html>, <head>, and <body> tags.
4. In the <head>, add metadata, a page title, and link the external CSS file (styles.css).
5. Inside the <body>, create a header section containing the department and college names.
6. Add a main section containing:
   - Welcome messages.
   - A link button redirecting to the college website.
   - An algorithm explanation section (Aim, Algorithm steps, Result).
7. Add a footer section with copyright.
8. Create a CSS file (styles.css).
9. Style the page by:
   - Setting the background color of the entire page to light blue.
   - Using Flexbox to center all content vertically and horizontally.
   - Styling header with dark blue background and white text.
   - Styling the link button with colors and hover effects.
   - Styling the algorithm section with border, padding, and background for readability.
   - Styling footer text.
10. Save both files and open index.html in a web browser.
11. The webpage loads with the content centered and styled as specified.
12. End

webpage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>CSE Department - St. Joseph's College of Engineering</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header>
    <h1>Department of Computer Science and Engineering</h1>
    <h2>St. Joseph's College of Engineering, OMR, Chennai</h2>
  </header>
  <main>
    <p>Welcome to the Department of Computer Science and Engineering at St. Joseph's College of
Engineering.</p>
```

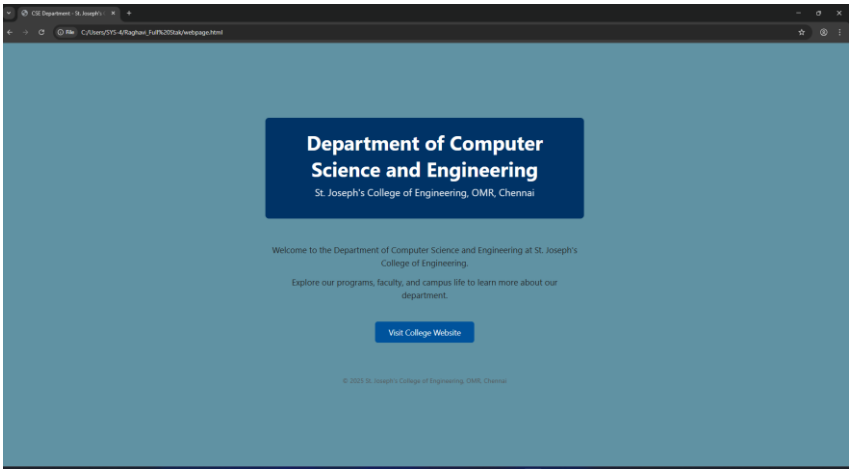
```
    <p>Explore our programs, faculty, and campus life to learn more about our department.</p>
    <a href="https://stjosephs.ac.in/index.html" target="_blank" class="btn-link">Visit College Website</a>
  </main>
  <footer>
    <p>© 2025 St. Joseph's College of Engineering, OMR, Chennai</p>
  </footer>
</body>
</html>
```

Style.css

```
body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #6092a3; /* Light blue background */
  margin: 0;
  height: 100vh;
  display: flex;
  justify-content: center;  /* Horizontal center */
  align-items: center;      /* Vertical center */
  text-align: center;
  color: #222;
  padding: 20px;
  box-sizing: border-box;
  flex-direction: column;
}
header {
  background-color: #003366;
  color: white;
  padding: 25px 10px;
  border-radius: 8px;
  margin-bottom: 40px;
  width: 100%;
  max-width: 700px;
}
header h1 {
  margin: 0;
  font-size: 2.8rem;
}
header h2 {
  margin-top: 8px;
  font-weight: 400;
  font-size: 1.4rem;
  opacity: 0.9;
}
main {
  max-width: 700px;
  width: 100%;
```

```
}
main p {
  font-size: 1.2rem;
  margin: 15px 0;
  line-height: 1.5;
}
.btn-link {
  display: inline-block;
  margin-top: 30px;
  padding: 12px 30px;
  background-color: #00539C;
  color: white;
  text-decoration: none;
  font-size: 1.1rem;
  border-radius: 6px;
  transition: background-color 0.3s ease;
}
.btn-link:hover {
  background-color: #003d66;
  cursor: pointer;
}
footer {
  margin-top: 60px;
  font-size: 0.9rem;
  color: #555;
  width: 100%;
  max-width: 700px;
}
```

Output



Result

Thus, a simple webpage has been created successfully using HTML and CSS.

Exp No: 2	**Build a chat module using HTML, CSS, and JavaScript**
Date:	

Aim

To create a basic chatbot that accepts user input, displays it, and responds with predefined replies.

Algorithm

1. Start
2. Display chat interface with input box and send button.
3. When user types a message and clicks “Send” or presses Enter:
   - Read the user input.
   - If input is empty, do nothing.
   - Display the user's message in the chat window.
4. Process the user message:
   - Convert the message to lowercase for comparison.
   - Check for keywords such as "hello", "how are you", "bye".
   - Based on the keyword, generate a predefined response.
   - If no keywords matched, reply with a generic message like "You said: [user message]".
5. Display the bot's response in the chat window.
6. Scroll the chat window to show the latest messages.
7. Repeat from step 3 for new user inputs.
8. End

Chat.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Chat Module</title>
  <link rel="stylesheet" href="style1.css">
</head>
<body>
  <div class="chat-container">
    <div class="chat-header">Simple Chat</div>
    <div class="chat-messages" id="chat-messages"></div>
    <div class="chat-input">
      <input type="text" id="user-input" placeholder="Type a message...">
      <button onclick="sendMessage()">Send</button>
    </div>
  </div>
  <script src="chatjs.js"></script>
</body>
</html>
```

Chatjs.js

```
function sendMessage() {
```

```
  const input = document.getElementById("user-input");
  const message = input.value.trim();
  if (message === "") return;
  addMessage(message, "user");
  // Clear input field
  input.value = "";
  // Simulate bot response
  setTimeout(() => {
    const botReply = getBotReply(message);
    addMessage(botReply, "bot");
  }, 500);
}
function addMessage(text, sender) {
  const chatMessages = document.getElementById("chat-messages");
  const msgDiv = document.createElement("div");
  msgDiv.classList.add("chat-message", sender);
  msgDiv.textContent = text;
  chatMessages.appendChild(msgDiv);
  chatMessages.scrollTop = chatMessages.scrollHeight; // auto-scroll
}
function getBotReply(userMessage) {
  // Very basic simulated responses
  if (userMessage.toLowerCase().includes("hello")) return "hi there";
  if (userMessage.toLowerCase().includes("help me learn html")) return "Yeah sure!";
  if (userMessage.toLowerCase().includes("bye")) return "Happy Learning!";
  return "You said: " + userMessage;
}
```

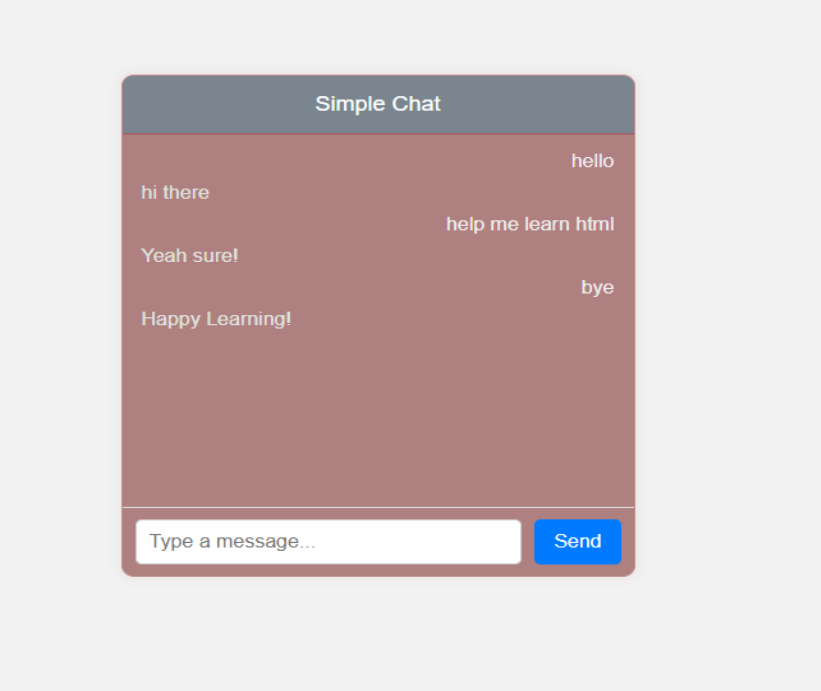
Style1.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
.chat-container {
  width: 400px;
  border: 1px solid #c08d8d;
  border-radius: 10px;
  background-color: rgb(175, 128, 128);
  display: flex;
  flex-direction: column;
  overflow: hidden;
```

```
  box-shadow: 0 0 10px rgba(160, 48, 48, 0.1);
}
.chat-header {
  background-color: #7b858f;
  color: white;
  padding: 15px;
  font-size: 18px;
  text-align: center;
}
.chat-messages {
  padding: 15px;
  height: 300px;
  overflow-y: auto;
  border-top: 1px solid #aa5555;
  border-bottom: 1px solid #ddd;
}
.chat-message {
  margin-bottom: 10px;
}
.chat-message.user {
  text-align: right;
  color: rgb(240, 240, 241);
}
.chat-message.bot {
  text-align: left;
  color: rgb(225, 230, 225);
}
.chat-input {
  display: flex;
  padding: 10px;
}
.chat-input input {
  flex: 1;
  padding: 10px;
  font-size: 16px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.chat-input button {
  padding: 10px 15px;
  margin-left: 10px;
  font-size: 16px;
  background-color: #007bff;
  color: white;
  border: none;
```

```
  border-radius: 5px;
  cursor: pointer;
}
.chat-input button:hover {
  background-color: #0056b3;
}
```

Output



Result

A simple Chat program has been created using HTML, JavaScript and CSS and executed successfully.

Exp No: 3	**Develop a calculator application using AngularJS**
Date:	

Aim

To create a calculator that performs basic arithmetic operations on two numbers.

Algorithm

1. Start.
2. Display input fields for two numbers and buttons for operations: Add, Subtract, Multiply, Divide.
3. User enters values in the two input fields.
4. When user clicks on an operation button:
    - Read the two input numbers.
    - Validate if both inputs are valid numbers.
        - If invalid, display an error message.
    - Perform the chosen operation:
        - Add: sum = num1 + num2
        - Subtract: result = num1 - num2
        - Multiply: result = num1 * num2
        - Divide:
            - If num2 is zero, display "Error: Division by zero".
            - Else result = num1 / num2
    - Display the result.
5. User can repeat steps 3 and 4 for new calculations.
6. End.

Calculator.html

```
<!DOCTYPE html>
<html ng-app="calcApp" lang="en">
<head>
 <meta charset="UTF-8" />
 <title>AngularJS Calculator</title>
 <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
 <style>
  /* Reset and base */
  * {
   box-sizing: border-box;
  }
  body {
   background: #282c34;
   display: flex;
   justify-content: center;
   align-items: center;
   height: 100vh;
   margin: 0;
   font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  }
```

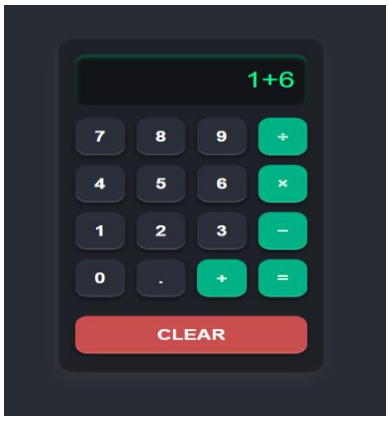
```
/* Calculator container */
.calculator {
  background: #1e2127;
  border-radius: 20px;
  box-shadow:
    0 10px 20px rgba(0, 0, 0, 0.4),
    inset 0 -4px 6px rgba(255, 255, 255, 0.1);
  width: 320px;
  padding: 25px 20px 30px;
  color: white;
  display: flex;
  flex-direction: column;
}
/* Display */
input[type="text"] {
  background: #121417;
  border-radius: 12px;
  border: none;
  padding: 20px 15px;
  font-size: 2.2rem;
  color: #00ff99;
  text-align: right;
  box-shadow: inset 0 4px 6px rgba(0, 255, 153, 0.3);
  margin-bottom: 20px;
  user-select: none;
  pointer-events: none; /* read-only style */
}
/* Buttons grid */
.button-row {
  display: flex;
  justify-content: space-between;
  margin-bottom: 15px;
}
button {
  flex: 1 1 22%;
  margin: 0 7px;
  height: 60px;
  border-radius: 15px;
  font-size: 1.5rem;
  font-weight: 600;
  color: white;
  background: #2a2f3a;
  border: none;
  box-shadow:
    0 4px 6px rgba(0,0,0,0.4),
```

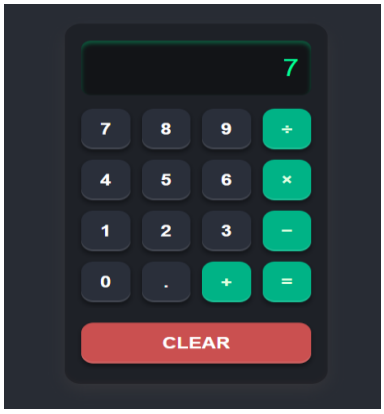
```
      inset 0 -3px 0 rgba(255, 255, 255, 0.1);
    cursor: pointer;
    transition: background 0.2s ease, box-shadow 0.2s ease;
    user-select: none;
   }
   /* Left and right margin fix for first and last buttons */
   button:first-child {
    margin-left: 0;
   }
   button:last-child {
    margin-right: 0;
   }
   button:hover {
    background: #00ff99;
    color: #1e2127;
    box-shadow:
      0 0 15px #00ff99,
      inset 0 4px 8px #00ff99;
   }
   /* Special buttons */
   button.operator {
    background: #00b386;
    color: #e0ffe0;
    font-weight: 700;
   }
   button.operator:hover {
    background: #00ff99;
    color: #1e2127;
    box-shadow:
      0 0 20px #00ff99,
      inset 0 6px 10px #00ff99;
   }
   button.clear {
    background: #cc3333;
    font-weight: 700;
   }
   button.clear:hover {
    background: #ff4d4d;
    box-shadow:
      0 0 15px #ff4d4d,
      inset 0 4px 8px #ff4d4d;
   }
  </style>
</head>
<body ng-controller="CalcController">
```

```
<div class="calculator">
  <input type="text" ng-model="expression" placeholder="0" readonly />
  <div class="button-row">
    <button ng-click="append('7')">7</button>
    <button ng-click="append('8')">8</button>
    <button ng-click="append('9')">9</button>
    <button ng-click="append('/')" class="operator">÷</button>
  </div>
  <div class="button-row">
    <button ng-click="append('4')">4</button>
    <button ng-click="append('5')">5</button>
    <button ng-click="append('6')">6</button>
    <button ng-click="append('*')" class="operator">×</button>
  </div>
  <div class="button-row">
    <button ng-click="append('1')">1</button>
    <button ng-click="append('2')">2</button>
    <button ng-click="append('3')">3</button>
    <button ng-click="append('-')" class="operator">−</button>
  </div>
  <div class="button-row">
    <button ng-click="append('0')">0</button>
    <button ng-click="append('.')">.</button>
    <button ng-click="append('+')" class="operator">+</button> <!-- Added addition -->
    <button ng-click="calculate()" class="operator">=</button>
  </div>
  <div style="margin-top: 15px; display: flex; justify-content: flex-end;">
    <button ng-click="clear()" class="clear" style="width: 100%;">C</button> <!-- Clear button full width
below -->
  </div>
</div>
<script>
  angular.module('calcApp', [])
    .controller('CalcController', ['$scope', function($scope) {
      $scope.expression = "";
      $scope.append = function(char) {
        const lastChar = $scope.expression.slice(-1);
        // Prevent two operators in a row
        if ('+-*/'.includes(char) && '+-*/'.includes(lastChar)) return;
        // Prevent multiple decimals in one number
        if (char === '.') {
          // Find last operator position
          const lastOperatorPos = Math.max(
            $scope.expression.lastIndexOf('+'),
            $scope.expression.lastIndexOf('-'),
```

```
          $scope.expression.lastIndexOf('*'),
          $scope.expression.lastIndexOf('/')
        );
        // If current number already has a '.', ignore
        if ($scope.expression.slice(lastOperatorPos + 1).includes('.')) return;
      }
      $scope.expression += char;
    };
    $scope.clear = function() {
      $scope.expression = "";
    };
    $scope.calculate = function() {
      try {
        $scope.expression = String(eval($scope.expression));
      } catch {
        $scope.expression = "Error";
      }
    };
  }]);
</script>
</body>
</html>
```

Output





Result

Thus, a program for calculator using HTML has been executed successfully.

Exp No: 4	Create a web application to manage the to-do list of users, where users can login and manage their to-do items using AngularJS
Date:	

Aim

To create an interactive todo list where users can add, mark complete/incomplete, delete todos, and see how many are completed.

Algorithm

1. Start.
2. Initialize todos = [], newTodo = "", and currentUser (e.g., { username: "..." }).
3. Show greeting: Hello, {{currentUser.username}}.
4. Render input box bound to newTodo and an **Add** button; also handle Enter key to trigger add.
5. addTodo() — if newTodo is non-empty, create todo = { text: newTodo, completed: false }, push to todos, then clear newTodo.
6. Render todos with ng-repeat="todo in todos track by $index".
7. For each todo show: a checkbox bound to todo.completed, the todo.text, and a **Delete** button.
8. Checkbox click updates todo.completed automatically via two-way binding.
9. deleteTodo(index) — remove todos.splice(index, 1) when Delete clicked.
10. Compute stats: completedCount = todos.filter(t => t.completed).length and total = todos.length.
11. Display stats as {{completedCount}} / {{total}} completed and await further add/toggle/delete interactions.
12. End.

Login.html

```
<div class="login-container">
  <h2>Login</h2>
  <form ng-submit="login()">
    <div class="form-group">
      <label for="username">Username</label>
      <input type="text" id="username" ng-model="user.username" required>
    </div>
    <div class="form-group">
      <label for="password">Password</label>
      <input type="password" id="password" ng-model="user.password" required>
    </div>
    <button type="submit">Login</button>
    <div class="error" ng-if="error">{{ error }}</div>
  </form>
</div>
```

authService.js

```
angular.module('todoApp')
.controller('AuthController', ['$scope', '$location', 'AuthService', function($scope, $location, AuthService) {
  $scope.user = {
    username: '',
```

```
        password: ''
    };
    $scope.login = function() {
        AuthService.login($scope.user)
        .then(function() {
            $location.path('/todos');
        }, function(error) {
            $scope.error = error.message;
        });
    };
}])
.service('AuthService', ['$q', '$timeout', function($q, $timeout) {
    var currentUser = null;
    // Mock users database
    var users = [
        { username: 'Raghavi', password: 'pass@123' },
        { username: 'user2', password: 'pass2' }
    ];
    this.login = function(user) {
        var deferred = $q.defer();
        $timeout(function() {
            var authenticatedUser = users.find(function(u) {
                return u.username === user.username && u.password === user.password;
            });
            if (authenticatedUser) {
                currentUser = authenticatedUser;
                deferred.resolve();
            } else {
                deferred.reject({ message: 'Invalid username or password' });
            }
        }, 500);
        return deferred.promise;
    };
    this.logout = function() {
        currentUser = null;
    };
    this.isLoggedIn = function() {
        return currentUser !== null;
    };
    this.getCurrentUser = function() {
        return currentUser;
    };
    this.ensureAuthenticated = function() {
        var deferred = $q.defer();
        if (this.isLoggedIn()) {
```

```
            deferred.resolve();
        } else {
            deferred.reject({ message: 'Not authenticated' });
        }
        return deferred.promise;
    };
}]);
```

Todo.html

```
<div class="todo-container">
    <h2>Welcome, {{ currentUser.username }}!</h2>
    <h3>Your Todo List</h3>
    <div class="add-todo">
        <input type="text" ng-model="newTodo" placeholder="Add a new todo..." ng-
keyup="$event.keyCode === 13 && addTodo()">
        <button ng-click="addTodo()">Add</button>
    </div>
    <ul class="todo-list">
        <li ng-repeat="todo in todos" ng-class="{completed: todo.completed}">
            <input type="checkbox" ng-model="todo.completed" ng-change="toggleTodo(todo)">
            <span>{{ todo.text }}</span>
            <button ng-click="deleteTodo(todo)">Delete</button>
        </li>
    </ul>
    <div class="stats" ng-if="todos.length">
        {{ todos.filter(function(t) { return t.completed; }).length }} of {{ todos.length }} completed
    </div>
</div>
```

todoService.js

```
angular.module('todoApp')
.controller('TodoController', ['$scope', 'AuthService', 'TodoService', function($scope, AuthService,
TodoService) {
    $scope.currentUser = AuthService.getCurrentUser();
    $scope.newTodo = '';
    $scope.todos = [];
    TodoService.getTodos($scope.currentUser.username)
    .then(function(todos) {
        $scope.todos = todos;
    });
    $scope.addTodo = function() {
        if ($scope.newTodo.trim() === '') return;
        var todo = {
            id: Date.now(),
            text: $scope.newTodo,
```

```
            completed: false,
            username: $scope.currentUser.username
        };
        TodoService.addTodo(todo)
        .then(function() {
            $scope.todos.push(todo);
            $scope.newTodo = '';
        });
    };
    $scope.toggleTodo = function(todo) {
        todo.completed = !todo.completed;
        TodoService.updateTodo(todo);
    };
    $scope.deleteTodo = function(todo) {
        TodoService.deleteTodo(todo.id)
        .then(function() {
            $scope.todos = $scope.todos.filter(function(t) {
                return t.id !== todo.id;
            });
        });
    };
}])
.service('TodoService', ['$q', '$timeout', function($q, $timeout) {
    var todos = [
        { id: 1, text: 'Learn AngularJS', completed: false, username: 'user1' },
        { id: 2, text: 'Build a todo app', completed: true, username: 'user1' },
        { id: 3, text: 'Deploy to production', completed: false, username: 'user2' }
    ];
    this.getTodos = function(username) {
        var deferred = $q.defer();
        $timeout(function() {
            var userTodos = todos.filter(function(todo) {
                return todo.username === username;
            });
            deferred.resolve(userTodos);
        }, 500);
        return deferred.promise;
    };
    this.addTodo = function(todo) {
        var deferred = $q.defer();
        $timeout(function() {
            todos.push(todo);
            deferred.resolve();
        }, 300);
        return deferred.promise;
```

```
    };
    this.updateTodo = function(todo) {
        var deferred = $q.defer();
        $timeout(function() {
            var index = todos.findIndex(function(t) {
                return t.id === todo.id;
            });
            if (index !== -1) {
                todos[index] = todo;
            }
            deferred.resolve();
        }, 300);
        return deferred.promise;
    };
    this.deleteTodo = function(id) {
        var deferred = $q.defer();
        $timeout(function() {
            todos = todos.filter(function(todo) {
                return todo.id !== id;
            });
            deferred.resolve();
        }, 300);
        return deferred.promise;
    };
}]);
```

app.js

```
angular.module('todoApp', ['ngRoute'])
.config(['$routeProvider', function($routeProvider) {
    $routeProvider
    .when('/login', {
        templateUrl: 'app/auth/login.html',
        controller: 'AuthController'
    })
    .when('/todos', {
        templateUrl: 'app/todo/todo.html',
        controller: 'TodoController',
        resolve: {
            auth: ['AuthService', function(AuthService) {
                return AuthService.ensureAuthenticated();
            }]
        }
    })
    .otherwise({
        redirectTo: '/login'
```

```
    });
}])
.run(['$rootScope', '$location', 'AuthService', function($rootScope, $location, AuthService) {
    $rootScope.$on('$routeChangeStart', function(event, next) {
        if (next.$$route && next.$$route.originalPath !== '/login' && !AuthService.isLoggedIn()) {
            event.preventDefault();
            $location.path('/login');
        }
    });
    $rootScope.isLoggedIn = function() {
        return AuthService.isLoggedIn();
    };
    $rootScope.logout = function() {
        AuthService.logout();
        $location.path('/login');
    };
}]);
```

Index.html

```
<!DOCTYPE html>
<html lang="en" ng-app="todoApp">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AngularJS Todo App</title>
    <link rel="stylesheet" href="styles.css">
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular-route.min.js"></script>
    <script src="app/app.js"></script>
    <script src="app/auth/authService.js"></script>
    <script src="app/todo/todoService.js"></script>
</head>
<body>
    <div ng-show="isLoggedIn()" class="logout-btn">
        <button ng-click="logout()">Logout</button>
    </div>
    <div ng-view></div>
</body>
</html>
```

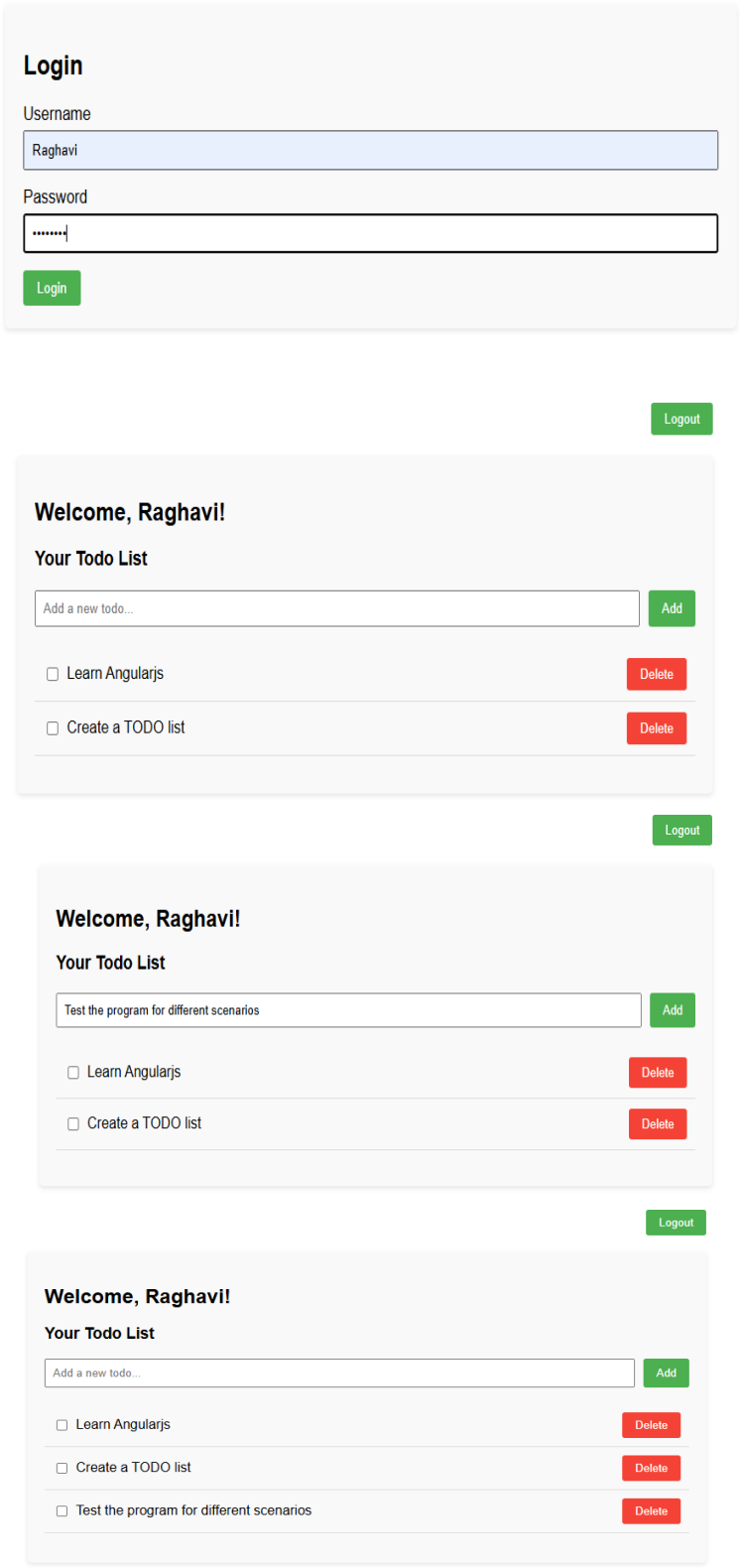
Style.css

```
body {
    font-family: Arial, sans-serif;
    max-width: 800px;
    margin: 0 auto;
```

```
    padding: 20px;
}
.logout-btn {
    text-align: right;
    margin-bottom: 20px;
}
.login-container, .todo-container {
    background: #f9f9f9;
    padding: 20px;
    border-radius: 5px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}
.form-group {
    margin-bottom: 15px;
}
.form-group label {
    display: block;
    margin-bottom: 5px;
}
.form-group input {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
}
button {
    padding: 8px 15px;
    background: #4CAF50;
    color: white;
    border: none;
    border-radius: 3px;
    cursor: pointer;
}
button:hover {
    background: #45a049;
}
.error {
    color: red;
    margin-top: 10px;
}
.todo-list {
    list-style: none;
    padding: 0;
}
.todo-list li {
    padding: 10px;
```

```
    border-bottom: 1px solid #ddd;
    display: flex;
    align-items: center;
}
.todo-list li.completed span {
    text-decoration: line-through;
    color: #888;
}
.todo-list input[type="checkbox"] {
    margin-right: 10px;
}
.todo-list button {
    margin-left: auto;
    background: #f44336;
}
.todo-list button:hover {
    background: #d32f2f;
}
.add-todo {
    display: flex;
    margin-bottom: 20px;
}
.add-todo input {
    flex-grow: 1;
    padding: 8px;
    margin-right: 10px;
}
.stats {
    margin-top: 20px;
    font-style: italic;
    color: #666;
}
```

Output



Result

A TO-DO list web application was successfully developed using AngularJS, allowing users to log in and manage their tasks (add, view, update, delete) securely.

Exp No: 5 Date:	Create a simple microblogging application (like Twitter) using React.js

Aim

To develop a simple micro-blogging web application using React JS that enables users to create and post content, follow other users, and view posts from the users they follow, thereby simulating core functionalities of platforms like Twitter.

Algorithm

1. Start.
2. Initialize React project and set up routing for Login, Register, Feed, and Profile pages.
3. Create user authentication system (sign up, login, logout) with state management (e.g., Context API or Redux).
4. Design a Post component to allow users to create and submit short text posts.
5. Store posts in a backend database (e.g., MongoDB / Firebase) with user IDs.
6. Build a Follow/Unfollow system by maintaining a list of followed user IDs for each account.
7. Fetch posts only from users that the current user follows and display them in the Feed.
8. Enable real-time updates (e.g., using WebSockets/Firebase listeners) so new posts appear instantly.
9. Implement a Profile page to show a user's posts and allow others to follow/unfollow.
10. Add likes/comments functionality to each post for interaction.
11. Optimize with pagination/infinite scroll and ensure responsive design for usability.
12. End.

File Structure (React App)

```
src/
├── components/
│   ├── Feed.js
│   ├── PostForm.js
│   └── UserList.js
├── App.js
└── index.js
```

src/App.js

```
import React, { useState } from 'react';
import PostForm from './components/PostForm';
import Feed from './components/Feed';
import UserList from './components/UserList';
const dummyUsers = ['alice', 'bob', 'charlie'];
function App() {
  const [currentUser] = useState('alice');
  const [posts, setPosts] = useState([]);
  const [following, setFollowing] = useState([]);
  const addPost = (content) => {
    const newPost = { user: currentUser, content };
    setPosts([newPost, ...posts]);
```

```
  };
  const toggleFollow = (user) => {
    if (following.includes(user)) {
      setFollowing(following.filter((u) => u !== user));
    } else {
      setFollowing([...following, user]);
    }
  };
  return (
    <div style={{ padding: '20px', fontFamily: 'Arial' }}>
      <h2>Micro-Blog (Logged in as @{currentUser})</h2>
      <UserList
        users={dummyUsers}
        currentUser={currentUser}
        following={following}
        onToggleFollow={toggleFollow}
      />
      <hr />
      <PostForm onPost={addPost} />
      <Feed posts={posts} following={following} />
    </div>
  );
}
export default App;
```

src/components/PostForm.js

```
import React, { useState } from 'react';
function PostForm({ onPost }) {
  const [content, setContent] = useState('');
  const handleSubmit = (e) => {
    e.preventDefault();
    if (content.trim()) {
      onPost(content);
      setContent('');
    }
  };
  return (
    <form onSubmit={handleSubmit}>
      <textarea
        value={content}
        onChange={(e) => setContent(e.target.value)}
        placeholder="What's on your mind?"
        rows={3}
        cols={50}
      />
```

```
      <br />
      <button type="submit">Post</button>
    </form>
  );
}
export default PostForm;
```

src/components/Feed.js

```
import React from 'react';
function Feed({ posts, following }) {
  const filteredPosts = posts.filter((post) => following.includes(post.user));
  return (
    <div>
      <h3>Feed</h3>
      {filteredPosts.length === 0 ? (
        <p>No posts from followed users.</p>
      ) : (
        filteredPosts.map((post, idx) => (
          <div key={idx} style={{ border: '1px solid #ccc', marginBottom: '10px', padding: '10px' }}>
            <strong>@{post.user}</strong>
            <p>{post.content}</p>
          </div>
        ))
      )}
    </div>
  );
}
export default Feed;
```

src/components/UserList.js

```
import React from 'react';
function UserList({ users, currentUser, following, onToggleFollow }) {
  return (
    <div>
      <h3>Users</h3>
      {users
        .filter((user) => user !== currentUser)
        .map((user) => (
          <div key={user}>
            @{user}
            <button
              onClick={() => onToggleFollow(user)}
              style={{ marginLeft: '10px' }}
            >
              {following.includes(user) ? 'Unfollow' : 'Follow'}
```

```
            </button>
          </div>
        ))}
      </div>
    );
  }
  export default UserList;
```

To Run This Project

1. Create a React app:
   npx create-react-app micro-blog
   cd micro-blog
2. Replace src/App.js and add the components folder with the 3 files above.
3. Start the app:
   npm start

Output



Result

The micro-blogging web app was successfully developed using ReactJS. Users can create posts, follow or unfollow others, and view posts only from the users they follow. The app works smoothly and shows basic features like those in Twitter.

Exp No: 6 Date:	Create a food delivery website using React.js

Aim

To develop a food delivery website using React JS that enables users to browse through a list of restaurants, view their menus, and place food orders from selected restaurants, providing a smooth, responsive, and user-friendly online food ordering experience.

Algorithm

1. Start.
2. Create a new React app: npx create-react-app food-delivery and scaffold pages/components.
3. Create three main components: RestaurantList, Menu, and Cart.
4. In App.js define initial restaurant data (each restaurant has id, name, and menu with items {id,name,price}).
5. Use useState to hold selectedRestaurant (null or restaurant id/object) and cart (array of items).
6. Render RestaurantList; bind each restaurant's View Menu button to onSelect that sets selectedRestaurant.
7. When selectedRestaurant is set render Menu for that restaurant and show each item with an Add to Cart button.
8. Implement handleAddToCart(item) to update cart (setCart(prev => [...prev, item])) and optionally dedupe/increment quantity.
9. Provide a back control (← Back to Restaurants) that sets selectedRestaurant back to null.
10. Render Cart showing each item and compute total with cart.reduce((s,i)=>s+i.price,0) and show Place Order only if cart.length>0.
11. Add basic CSS spacing/borders, start dev server npm start, and test: list → menu → add to cart → view total → place order.
12. End.

Folder Structure

```
food-delivery/
├── public/
│   └── index.html
└── src/
    ├── components/
    │   ├── RestaurantList.js
    │   ├── Menu.js
    │   └── Cart.js
    ├── App.js
    └── index.js
```

src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(<App />);
```

src/App.js

```
import React, { useState } from 'react';
import RestaurantList from './components/RestaurantList';
import Menu from './components/Menu';
import Cart from './components/Cart';
const App = () => {
 const [selectedRestaurant, setSelectedRestaurant] = useState(null);
 const [cart, setCart] = useState([]);
 const restaurants = [
  { id: 1, name: "Pizza Palace", menu: [{ id: 1, name: "Pepperoni Pizza", price: 299 }, { id: 2, name:
   "Veggie Pizza", price: 249 }] },
  { id: 2, name: "Burger Barn", menu: [{ id: 3, name: "Cheese Burger", price: 199 }, { id: 4, name: "Veg
   Burger", price: 149 }] },
 ];
 const handleAddToCart = (item) => {
  setCart([...cart, item]);
 };
 return (
  <div style={{ padding: "20px" }}>
   <h1>Food Delivery App</h1>
   {!selectedRestaurant ? (
    <RestaurantList restaurants={restaurants} onSelect={setSelectedRestaurant} />
   ) : (
    <>
     <button onClick={() => setSelectedRestaurant(null)}>← Back to Restaurants</button>
     <Menu menu={selectedRestaurant.menu} onAddToCart={handleAddToCart} />
    </>
   )}
   <Cart items={cart} />
  </div>
 );
};
export default App;
```

src/components/RestaurantList.js

```
import React from 'react';
const RestaurantList = ({ restaurants, onSelect }) => {
 return (
  <div>
   <h2>Restaurants</h2>
   {restaurants.map((res) => (
    <div key={res.id}>
     <h3>{res.name}</h3>
```

```
        <button onClick={() => onSelect(res)}>View Menu</button>
       </div>
     ))}
   </div>
  );
};
export default RestaurantList;
```

src/components/Menu.js

```
import React from 'react';
const Menu = ({ menu, onAddToCart }) => {
  return (
    <div>
      <h2>Menu</h2>
      {menu.map((item) => (
        <div key={item.id}>
          <p>{item.name} - ₹{item.price}</p>
          <button onClick={() => onAddToCart(item)}>Add to Cart</button>
        </div>
      ))}
    </div>
  );
};
export default Menu;
```

src/components/Cart.js

```
import React from 'react';
const Cart = ({ items }) => {
  const total = items.reduce((sum, item) => sum + item.price, 0);
  return (
    <div style={{ marginTop: "20px", borderTop: "1px solid #ccc" }}>
      <h2>Cart</h2>
      {items.map((item, index) => (
        <p key={index}>{item.name} - ₹{item.price}</p>
      ))}
      <h3>Total: ₹{total}</h3>
      {items.length > 0 && <button>Place Order</button>}
    </div>
  );
};
export default Cart;
```

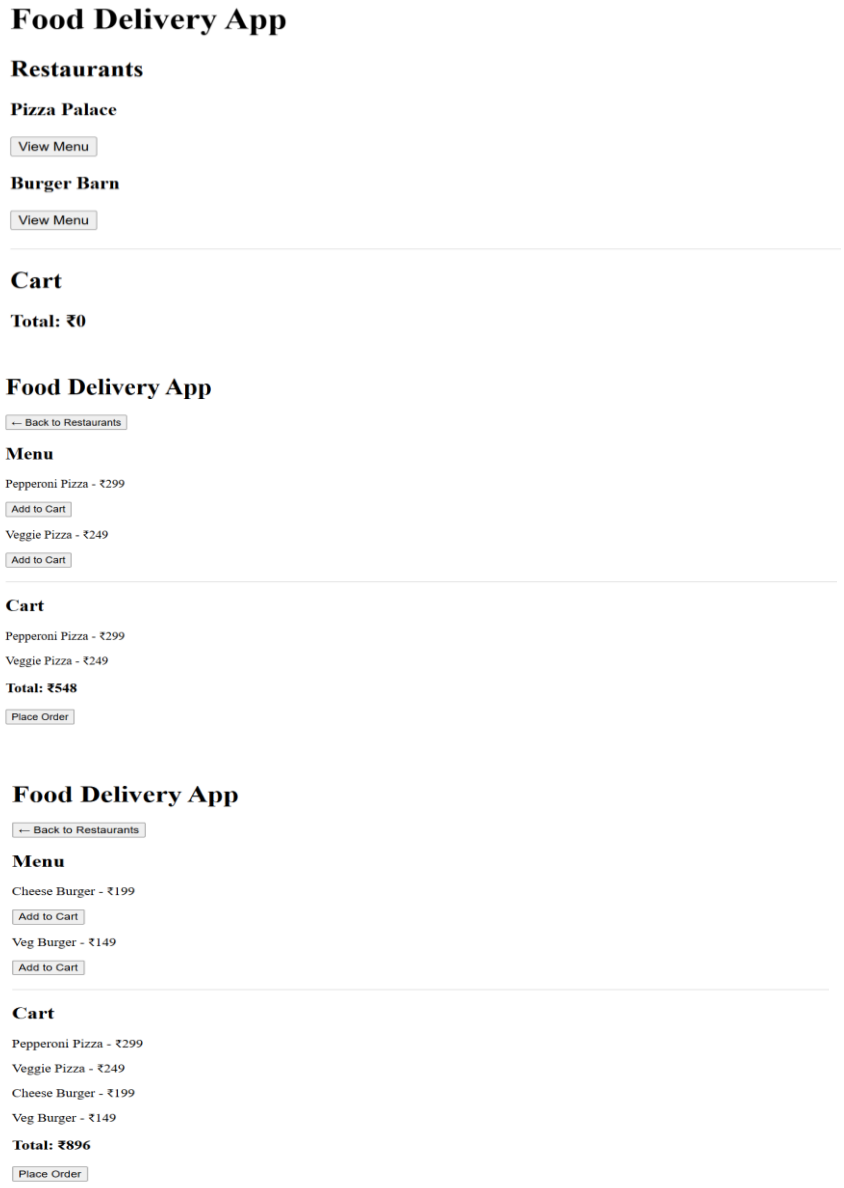
How to Run

Create a new React app:

```
npx create-react-app food-delivery
```

cd food-delivery
Replace src content with the files above.
Start the server:
npm start

Output



Result

The Food Delivery website was successfully created using React JS. Users can easily browse restaurants, view their menus, and place food orders. This app provides a smooth user experience for online food ordering.

Exp No: 7	**Build web application for shopping cart**
Date:	

Aim

To Build web application for shopping cart with registration, login, catalogue and cart pages using Node.js.

Algorithm

1. Start.
2. create backend/ (Express) and public/ (static HTML/CSS/JS) folders; install express, mongoose, bcryptjs, jsonwebtoken, cors, and load .env (PORT, MONGO_URI, JWT_SECRET).
3. define Mongoose models — User{ name,email,password }, Product{ title,desc,price,image }, CartItem{ userRef, productRef, quantity }.
4. configure Express to serve public/ (static HTML, CSS, client JS) and expose API routes under /api/*.
5. implement /api/auth/register to validate input, hash password with bcrypt, save user, and return JWT; /api/auth/login to verify password and return JWT + user info.
6. create middleware that reads Authorization: Bearer <token>, verifies JWT, and attaches req.user for protected routes.
7. implement /api/products (GET list), and optional /api/products/seed (POST) to insert sample products.
8. implement protected endpoints /api/cart (GET user cart), /api/cart/add (POST productId, qty), /api/cart/update (POST cartItemId, qty), /api/cart/remove (POST cartItemId); persist via CartItem collection.
9. build public/index.html (catalogue), public/auth.html (register/login), public/cart.html (cart page) with simple forms and minimal CSS; include a public/app.js that handles DOM, fetch/XHR, localStorage for JWT/user.
10. client JS sends form data to /api/auth/*, stores JWT in localStorage, sends Authorization header on protected cart calls, updates DOM after responses (add/remove/qty).
11. start MongoDB, run node server.js (or nodemon), open http://localhost:PORT/ in browser, seed products, register/login, add items to cart and verify DB records and UI updates.
12. End.

Folder Structure

```
shopping-cart-app/
├── public/
│   ├── css/
│   │   └── styles.css
│   ├── js/
│   │   └── main.js
│   └── images/      (optional for product images)
├── views/
│   ├── login.html
│   ├── register.html
│   ├── catalogue.html
│   └── cart.html
├── server.js
└── package.json
```

package.json

```
{
 "name": "shopping-cart-app",
 "version": "1.0.0",
 "main": "server.js",
 "scripts": {
  "start": "node server.js"
 },
 "dependencies": {
  "express": "^4.18.2",
  "express-session": "^1.17.3",
  "body-parser": "^1.20.2"
 } }
```

server.js (Core Node.js Logic)

```
const express = require('express');
const session = require('express-session');
const bodyParser = require('body-parser');
const path = require('path');
const app = express();
const PORT = 3000;
app.use(express.static('public'));
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
app.use(session({
 secret: 'secretKey',
 resave: false,
 saveUninitialized: true }));
// In-memory users and products
const users = {};
const products = [
 { id: 1, name: 'Laptop', price: 500 },
 { id: 2, name: 'Headphones', price: 100 },
 { id: 3, name: 'Keyboard', price: 80 }];
// Routes
app.get('/', (req, res) => {
 res.redirect('/login'); });
app.get('/login', (req, res) => {
 res.sendFile(path.join(__dirname, 'views/login.html')); });
app.get('/register', (req, res) => {
 res.sendFile(path.join(__dirname, 'views/register.html')); });
app.post('/register', (req, res) => {
 const { username, password } = req.body;
 if (users[username]) return res.send('User already exists');
 users[username] = { password, cart: [] };
```

```
  res.redirect('/login'); });
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  if (users[username]?.password === password) {
    req.session.user = username;
    res.redirect('/catalogue');
  } else {
    res.send('Invalid credentials');
  } });
app.get('/catalogue', (req, res) => {
  if (!req.session.user) return res.redirect('/login');
  res.sendFile(path.join(__dirname, 'views/catalogue.html')); });
app.get('/products', (req, res) => {
  if (!req.session.user) return res.status(401).send('Unauthorized');
  res.json(products); });
app.post('/add-to-cart', (req, res) => {
  const { productId } = req.body;
  const username = req.session.user;
  const product = products.find(p => p.id === parseInt(productId));
  if (product) users[username].cart.push(product);
  res.sendStatus(200); });
app.get('/cart', (req, res) => {
  if (!req.session.user) return res.redirect('/login');
  res.sendFile(path.join(__dirname, 'views/cart.html')); });
app.get('/cart-items', (req, res) => {
  const username = req.session.user;
  res.json(users[username]?.cart || []); });
app.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));
```

views/login.html

```
<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
  <h2>Login</h2>
  <form action="/login" method="post">
    Username: <input name="username" required><br>
    Password: <input type="password" name="password" required><br>
    <button type="submit">Login</button>
  </form>
  <p>Don't have an account? <a href="/register">Register</a></p>
</body>
</html>
```

views/register.html

```
<!DOCTYPE html>
<html>
<head><title>Register</title></head>
<body>
  <h2>Register</h2>
  <form action="/register" method="post">
    Username: <input name="username" required><br>
    Password: <input type="password" name="password" required><br>
    <button type="submit">Register</button>
  </form>
  <p>Already have an account? <a href="/login">Login</a></p>
</body>
</html>
```

views/catalogue.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Catalogue</title>
  <script defer src="/js/main.js"></script>
</head>
<body>
  <h2>Product Catalogue</h2>
  <div id="product-list"></div>
  <a href="/cart">Go to Cart</a>
</body>
</html>
```

views/cart.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Cart</title>
  <script defer src="/js/main.js"></script>
</head>
<body>
  <h2>Your Cart</h2>
  <div id="cart-list"></div>
  <a href="/catalogue">Back to Catalogue</a>
</body>
</html>
```

public/js/main.js

```
document.addEventListener('DOMContentLoaded', () => {
  if (location.pathname === '/catalogue') {
    fetch('/products')
      .then(res => res.json())
      .then(products => {
        const list = document.getElementById('product-list');
        products.forEach(p => {
          const item = document.createElement('div');
          item.innerHTML = `
            <strong>${p.name}</strong> - $${p.price}
            <button onclick="addToCart(${p.id})">Add to Cart</button>;
          list.appendChild(item);        });        });  }
  if (location.pathname === '/cart') {
    fetch('/cart-items')
      .then(res => res.json())
      .then(items => {
        const list = document.getElementById('cart-list');
        if (items.length === 0) {
          list.textContent = 'Cart is empty.';
        } else {
          items.forEach(p => {
            const item = document.createElement('div');
            item.textContent = `${p.name} - $${p.price}`;
            list.appendChild(item);
          });        }        });   }   });
function addToCart(productId) {
  fetch('/add-to-cart', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ productId })
  }).then(() => alert('Added to cart!'));
}
```

public/css/styles.css (optional starter)

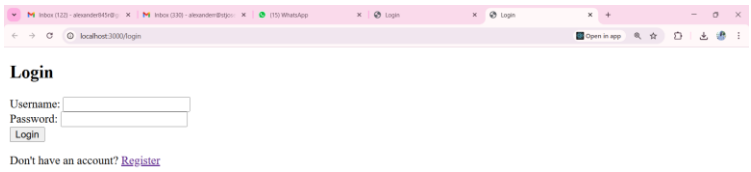
```
body {
  font-family: Arial, sans-serif;
  padding: 20px;
}
h2 {
  color: #333;
}
button {
  margin-left: 10px;
  padding: 5px 10px;
```

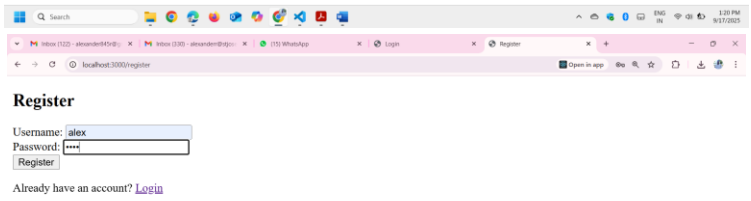
}

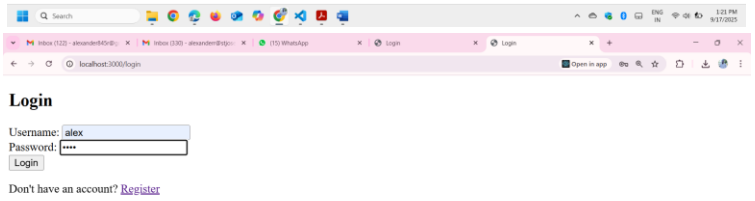
How to Run

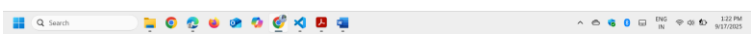
1. Initialize project
npm install
2. Start server
npm start
3. Open in browser
http://localhost:3000

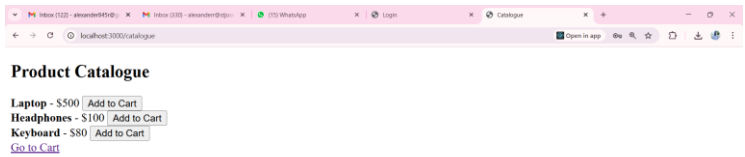
Output

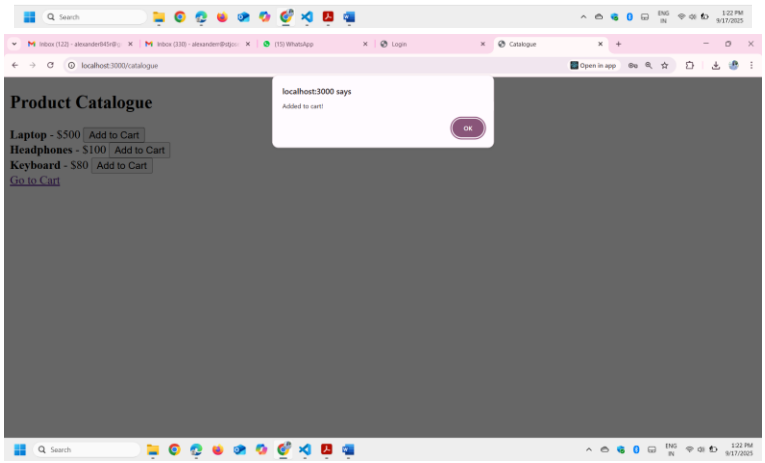


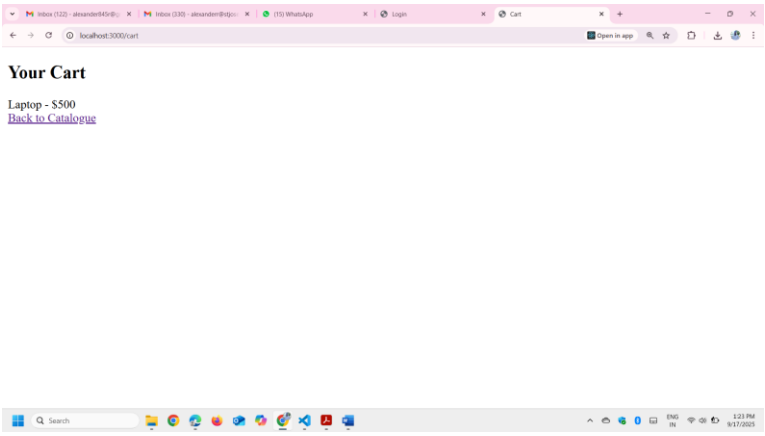












Result

The Shopping Cart web application was successfully developed using **Node.js, HTML, and CSS**. Users can easily register, login, browse the product catalogue, and manage their shopping cart by adding, updating, or removing items. The application provides a smooth user experience for online shopping with secure authentication and persistent cart management.

Exp No: 8 Date:	Develop a leave management system

Aim

To develop a full stack Leave Management System for an organization where users can apply for different types of leaves such as Casual Leave and Medical Leave, and view their available leave balances, using modern web development tools and MongoDB as the backend database.

Algorithm

1. Start.
2. create backend/ (Express) and public/ (static HTML/CSS/JS) folders; install express, mongoose, bcryptjs, jsonwebtoken, cors, dotenv and load .env (PORT, MONGO_URI, JWT_SECRET).
3. define Mongoose models — User { name, email, password }, Leave { userRef, type, startDate, endDate, days, status, appliedAt }, LeaveBalance { userRef, casualDays, medicalDays }.
4. configure Express to serve public/ (static HTML/CSS/client JS) and expose API routes under /api/* (auth, leaves, balances).
5. implement /api/auth/register to validate input, hash password with bcrypt, create LeaveBalance (initial quotas), save user and return JWT; /api/auth/login to verify password and return JWT + user info.
6. create auth middleware that reads Authorization: Bearer <token>, verifies JWT, attaches req.user for protected routes, and handles expired/invalid tokens.
7. implement /api/leaves (GET user leaves), /api/leaves/apply (POST type, startDate, endDate → calculate days), /api/leaves/:id (GET single) and /api/leaves/cancel (POST id) — protect apply/cancel routes.
8. implement server-side logic in /api/leaves/apply: compute days, check LeaveBalance for requested type, if sufficient → create Leave with status: "Approved" (or Pending if you want approval flow) and decrement balance; else return 400 with message.
9. implement /api/balances (GET current user balances) and optional /api/balances/seed (POST) to set initial quotas (e.g., casual:12, medical:10) for testing.
10. build public pages: public/index.html (dashboard showing balances & apply form), public/auth.html (register/login), public/leaves.html (list applied leaves & cancel), plus public/app.js that handles DOM, fetch/XHR, and localStorage for JWT/user.
11. client JS behavior: send auth forms to /api/auth/*, store JWT in localStorage, include Authorization header on protected /api/* calls, show balances from /api/balances, send apply requests to /api/leaves/apply, update DOM after responses and show error/success messages.
12. run MongoDB, node server.js (or nodemon), open http://localhost:PORT/, seed balances (if needed), register/login, apply/cancel leaves, and verify DB records + UI updates.
13. End.

Project Structure

```
leave-management-system/
├── backend/
│   ├── models/
│   │   └── Leave.js
│   ├── routes/
│   │   └── leaveRoutes.js
│   └── server.js
```

```
├── frontend/
│   └── index.html
├── package.json
```

Backend Setup

models/Leave.js

```
const mongoose = require('mongoose');
const leaveSchema = new mongoose.Schema({
  employeeId: String,
  type: { type: String, enum: ['Casual', 'Medical'] },
  days: Number,
  appliedDate: { type: Date, default: Date.now }
});
module.exports = mongoose.model('Leave', leaveSchema);
```

routes/leaveRoutes.js

```
const express = require('express');
const router = express.Router();
const Leave = require('../models/Leave');
const defaultLeave = { Casual: 12, Medical: 8 };
// Apply for leave
router.post('/apply', async (req, res) => {
  const { employeeId, type, days } = req.body;
  // Save leave request
  const leave = new Leave({ employeeId, type, days });
  await leave.save();
  res.json({ message: 'Leave applied successfully' });
});
// Get leave balance
router.get('/balance/:employeeId', async (req, res) => {
  const { employeeId } = req.params;

  const leaves = await Leave.find({ employeeId });
  let used = { Casual: 0, Medical: 0 };
  leaves.forEach(l => used[l.type] += l.days);
  res.json({
    Casual: defaultLeave.Casual - used.Casual,
    Medical: defaultLeave.Medical - used.Medical
  });
});
module.exports = router;
```

server.js

```
const express = require('express');
const mongoose = require('mongoose');
```

```
const cors = require('cors');
const app = express();
app.use(cors());
app.use(express.json());
mongoose.connect('mongodb://localhost:27017/leaveDB', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => console.log('MongoDB Connected'));
const leaveRoutes = require('./routes/leaveRoutes');
app.use('/api/leaves', leaveRoutes);
app.listen(5000, () => console.log('Server running on port 5000'));
```

Frontend

frontend/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Leave Management</title>
</head>
<body>
  <h2>Apply for Leave</h2>
  <form id="leaveForm">
    <input type="text" id="empId" placeholder="Employee ID" required>
    <select id="type">
      <option value="Casual">Casual</option>
      <option value="Medical">Medical</option>
    </select>
    <input type="number" id="days" placeholder="Days" required>
    <button type="submit">Apply</button>
  </form>
  <h2>Check Leave Balance</h2>
  <input type="text" id="checkId" placeholder="Employee ID">
  <button onclick="checkBalance()">Check</button>
  <div id="result"></div>
  <script>
    document.getElementById('leaveForm').addEventListener('submit', async e => {
      e.preventDefault();
      const res = await fetch('http://localhost:5000/api/leaves/apply', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          employeeId: document.getElementById('empId').value,
          type: document.getElementById('type').value,
          days: document.getElementById('days').value
```

```
      })
    });
    const data = await res.json();
    alert(data.message);
  });
  async function checkBalance() {
    const id = document.getElementById('checkId').value;
    const res = await fetch(`http://localhost:5000/api/leaves/balance/${id}`);
    const data = await res.json();
    document.getElementById('result').innerText =
      `Casual: ${data.Casual} | Medical: ${data.Medical}`;
  }
 </script>
</body>
</html>
```

Run the Application

1. Start MongoDB

mongod

2. Run Backend

cd backend

npm init -y

npm install express mongoose cors

node server.js

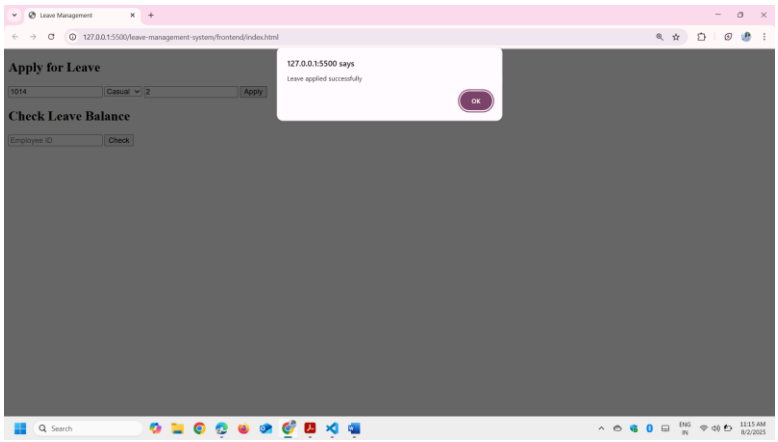
3. Open Frontend

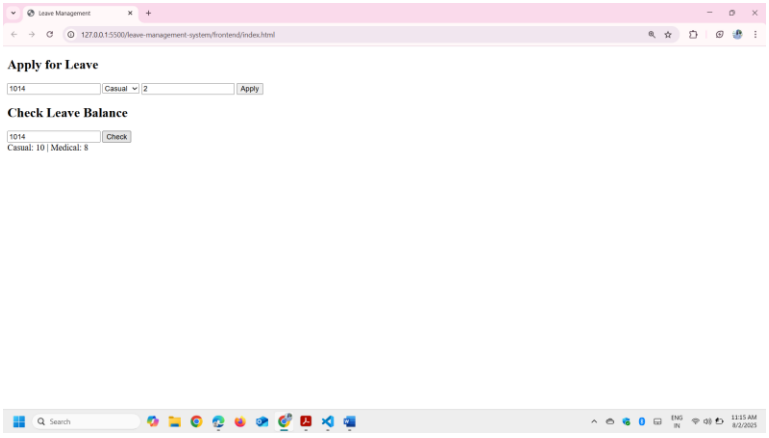
Just open frontend/index.html in a browser.

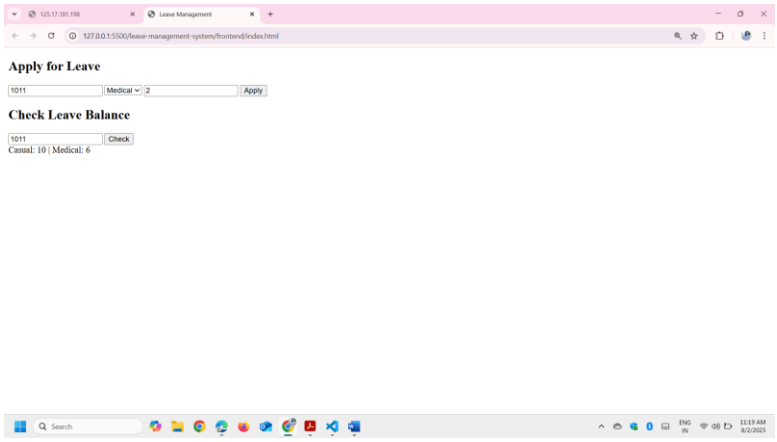
Output











Result

The Leave Management System web application was successfully developed using Node.js, Express, MongoDB, HTML, and CSS. Users can securely register, login, and apply for different types of leaves such as casual and medical leave. The application provides a smooth and secure user experience with authentication and persistent leave data management.

Exp No: 9	**Develop a simple dashboard for project management**
Date:	

Aim

To develop a simple dashboard for project management where the status of various tasks are available. New tasks can be added and the status of existing tasks can be changed among Pending, In Progress or Completed using full stack development tools and MongoDB.

Algorithm

1. Start.
2. Connect to MongoDB and load environment variables (PORT, MONGO_URI, JWT_SECRET).
3. Create backend folder with Express and install express, mongoose, cors, dotenv, bcryptjs, jsonwebtoken.
4. Define Mongoose models — User {name,email,password}, Task {title,description,assignee,status,createdAt,dueDate}.
5. Configure Express to serve public/ (static HTML/CSS/JS) and expose API routes under /api/*.
6. Implement auth routes /api/auth/register and /api/auth/login (hash passwords, return JWT); add auth middleware to protect task routes.
7. Implement task endpoints: GET /api/tasks (list/filter), POST /api/tasks (create), PUT /api/tasks/:id (update fields/status), DELETE /api/tasks/:id (optional).
8. Server logic: validate input on create/update, enforce allowed status values (Pending, In Progress, Completed), and save changes to MongoDB.
9. Build frontend pages public/index.html (dashboard showing task cards/filters), public/auth.html (login/register) and public/app.js to call APIs, store JWT, and update the DOM.
10. Run MongoDB, start server (node server.js or nodemon), open http://localhost:PORT/, add tasks and change statuses
11. End.

Folder Structure

```
project-dashboard/
├── backend/
│   └── server.js
├── frontend/
│   └── (React App)
```

Backend Setup (Node.js + Express + MongoDB)

Create Backend Folder

```
mkdir project-dashboard
cd project-dashboard
mkdir backend
cd backend
npm init -y
npm install express mongoose cors body-parser
```

Create server.js

```
// backend/server.js
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();
const PORT = 5000;
app.use(cors());
app.use(express.json());
// MongoDB connection
mongoose.connect('mongodb://localhost:27017/project_dashboard', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Task schema
const taskSchema = new mongoose.Schema({
  title: String,
  status: {
    type: String,
    enum: ['Pending', 'In Progress', 'Completed'],
    default: 'Pending'
  }
});
const Task = mongoose.model('Task', taskSchema);
// Routes
// Get all tasks
app.get('/tasks', async (req, res) => {
  const tasks = await Task.find();
  res.json(tasks);
});
// Add new task
app.post('/tasks', async (req, res) => {
  const task = new Task(req.body);
  await task.save();
  res.json(task);
});
// Update task status
app.put('/tasks/:id', async (req, res) => {
  const { id } = req.params;
  const { status } = req.body;
  const task = await Task.findByIdAndUpdate(id, { status }, { new: true });
  res.json(task);
});
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);  });
```

Start MongoDB and Run Backend

Make sure MongoDB is running (e.g., mongod in terminal or MongoDB Compass service).
in VS Code terminal:
cd backend
node server.js

Frontend Setup (React)

Create React App

Open a new terminal in VS Code:
cd ..
npx create-react-app frontend
cd frontend
npm install axios

Create TaskDashboard.js

```
// frontend/src/TaskDashboard.js
import React, { useEffect, useState } from 'react';
import axios from 'axios';
const TaskDashboard = () => {
 const [tasks, setTasks] = useState([]);
 const [newTitle, setNewTitle] = useState('');
 const fetchTasks = async () => {
  const res = await axios.get('http://localhost:5000/tasks');
  setTasks(res.data);
 };
 const addTask = async () => {
  if (!newTitle) return;
  await axios.post('http://localhost:5000/tasks', { title: newTitle });
  setNewTitle('');
  fetchTasks();
 };
 const updateStatus = async (id, status) => {
  await axios.put(`http://localhost:5000/tasks/${id}`, { status });
  fetchTasks();
 };
 useEffect(() => {
  fetchTasks();
 }, []);
 return (
  <div style={{ padding: '2rem' }}>
   <h2>Project Task Dashboard</h2>
   <input
    placeholder="New Task Title"
    value={newTitle}
    onChange={(e) => setNewTitle(e.target.value)}
```

```
      />
      <button onClick={addTask}>Add Task</button>
      <ul>
        {tasks.map((task) => (
          <li key={task._id}>
            <strong>{task.title}</strong> - <em>{task.status}</em>
            <select
              value={task.status}
              onChange={(e) => updateStatus(task._id, e.target.value)}
            >
              <option>Pending</option>
              <option>In Progress</option>
              <option>Completed</option>
            </select>
          </li>
        ))}
      </ul>
    </div>
  );
};
export default TaskDashboard;
```

Use in App Component

Edit src/App.js:

```
import React from 'react';
import TaskDashboard from './TaskDashboard';
function App() {
  return (
    <div className="App">
      <TaskDashboard />
    </div>
  );
}
export default App;
```

Run React App

npm start

Open browser: http://localhost:3000

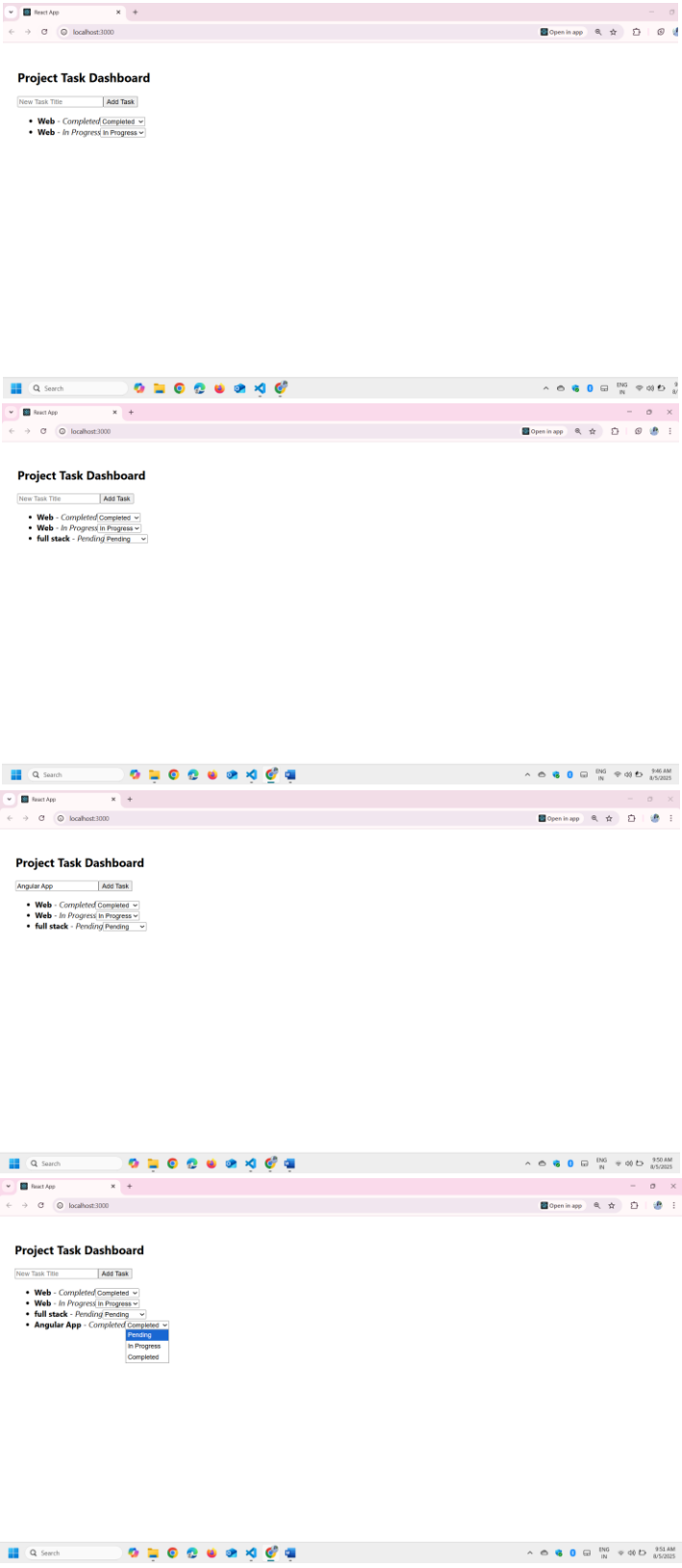
Final Result

Frontend: React dashboard on http://localhost:3000

Backend: Express server on http://localhost:5000

DB: MongoDB local connection (mongodb://localhost:27017)

Output



Result

The Project Management Dashboard web application was successfully developed using Node.js, Express, MongoDB, HTML, and CSS. Users can securely register, login, and manage tasks by adding new ones and updating their status among Pending, In Progress, or Completed.

Exp No: 10	**Develop an online survey application**
Date:	

Aim

To Develop an online survey application where a collection of questions is available and users are asked to answer any random 5 questions using full stack development tools and MongoDB.

Algorithm

1. Start.
2. Seed MongoDB with questions documents (text, type, choices) and start the Express server.
3. Expose GET /api/questions/random?count=5 and POST /api/responses (plus GET /api/responses for admin).
4. On GET /api/questions/random, use Question.aggregate([{ $sample: { size: count } }]) to pick random questions.
5. Return the random questions as JSON to the client.
6. Frontend fetches the 5 questions, renders radio inputs for choice and textareas for text.
7. User selects/enters answers; frontend builds payload { userId, answers:[{questionId, answer}] }.
8. Frontend POST the payload to /api/responses; show success/failure to user.
9. Backend validates payload, saves a response document with answers and createdAt.
10. Admin reads GET /api/responses (use .populate('answers.questionId', 'text')) to display question text with answers.
11. Dev/run: ensure Mongo running, set MONGO_URI and REACT_APP_API_URL, enable CORS, run backend and frontend in VS Code.
12. End.

File structure

```
survey-app/
├─ backend/
│  ├─ package.json
│  ├─ server.js
│  ├─ .env
│  ├─ models/Question.js
│  ├─ models/Response.js
│  └─ seed.js
└─ frontend/
   ├─ package.json
   └─ src/
      ├─ index.js
      ├─ App.js
      └─ components/Survey.js
```

Backend (Express + Mongoose)

Create backend/package.json

```
{
 "name": "survey-backend",
 "version": "1.0.0",
 "main": "server.js",
 "scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js",
  "seed": "node seed.js"
 },
 "dependencies": {
  "cors": "^2.8.5",
  "dotenv": "^16.0.0",
  "express": "^4.18.2",
  "mongoose": "^7.0.0"
 },
 "devDependencies": {
  "nodemon": "^2.0.0"
 }
}
```

Create backend/.env

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/surveyapp
```

(If you use MongoDB Atlas, the MONGO_URI will be the Atlas connection string.)

backend/models/Question.js:

```
const mongoose = require('mongoose');
const QuestionSchema = new mongoose.Schema({
 text: { type: String, required: true },
 type: { type: String, enum: ['text','choice'], default: 'text' },
 choices: { type: [String], default: [] } // used when type === 'choice'
});
module.exports = mongoose.model('Question', QuestionSchema);
```

backend/models/Response.js

```
const mongoose = require('mongoose');
const ResponseSchema = new mongoose.Schema({
 userId: { type: String, default: null }, // optional
 answers: [
  {
   questionId: { type: mongoose.Schema.Types.ObjectId, ref: 'Question' },
   answer: mongoose.Schema.Types.Mixed
```

```
    }
  ],
  createdAt: { type: Date, default: Date.now }
});
module.exports = mongoose.model('Response', ResponseSchema);
```

backend/server.js

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const Question = require('./models/Question');
const ResponseModel = require('./models/Response');
const app = express();
app.use(cors());
app.use(express.json());
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI;
mongoose.connect(MONGO_URI, { useNewUrlParser: true, useUnifiedTopology: true })
  .then(()=> console.log('MongoDB connected'))
  .catch(err => console.error('Mongo connection error', err));
// Endpoint: get N random questions (default 5)
app.get('/api/questions/random', async (req, res) => {
  try {
    const count = parseInt(req.query.count || '5', 10);
    // Use MongoDB aggregation sample for true random selection
    const questions = await Question.aggregate([{ $sample: { size: count } }]);
    res.json(questions);
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: 'Server error' });
  }
});
// Endpoint: submit responses
app.post('/api/responses', async (req, res) => {
  try {
    const { userId, answers } = req.body;
    const resp = new ResponseModel({ userId, answers });
    await resp.save();
    res.json({ ok: true, id: resp._id });
  } catch (err) {
    console.error(err);
    res.status(400).json({ ok: false, error: err.message });
  }
});
```

```
// Optional: create question (for admin/testing)
app.post('/api/questions', async (req, res) => {
  try {
    const q = new Question(req.body);
    await q.save();
    res.json(q);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
app.listen(PORT, () => console.log(`Server listening on ${PORT}`));
```

backend/seed.js — quick seeding script to populate some questions

```
require('dotenv').config();
const mongoose = require('mongoose');
const Question = require('./models/Question');
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/surveyapp';
const sampleQuestions = [
  { text: 'How satisfied are you with our product?', type: 'choice', choices: ['Very
satisfied','Satisfied','Neutral','Dissatisfied','Very dissatisfied'] },
  { text: 'How likely are you to recommend us to a friend?', type: 'choice', choices: ['Very
likely','Likely','Maybe','Unlikely','Very unlikely'] },
  { text: 'What feature would you like to see next?', type: 'text' },
  { text: 'How easy was it to sign up?', type: 'choice', choices: ['Very easy','Easy','Ok','Hard','Very hard'] },
  { text: 'Any other feedback?', type: 'text' },
  { text: 'Rate our customer support.', type: 'choice', choices: ['5','4','3','2','1'] },
  { text: 'Which platform do you use most?', type: 'choice', choices: ['Web','iOS','Android','Other'] },
  { text: 'How often do you use the product?', type: 'choice', choices:
['Daily','Weekly','Monthly','Rarely','Never'] }
];
async function seed() {
  await mongoose.connect(MONGO_URI);
  await Question.deleteMany({});
  await Question.insertMany(sampleQuestions);
  console.log('Seeded questions');
  mongoose.disconnect();
}
seed().catch(err => {
  console.error(err);
  mongoose.disconnect();
});
```

Frontend (React)

Use Create React App for simplicity.

From the frontend folder run npx create-react-app .

frontend/src/App.js

```
import React from 'react';
import Survey from './components/Survey';
function App(){
 return (
  <div style={{ maxWidth: 800, margin: '2rem auto', fontFamily: 'sans-serif' }}>
   <h1>Quick 5-Question Survey</h1>
   <Survey />
  </div>
 );
}
export default App;
```

frontend/src/components/Survey.js

```
import React, { useEffect, useState } from 'react';
const API = process.env.REACT_APP_API_URL || 'http://localhost:5000';
export default function Survey(){
 const [questions, setQuestions] = useState([]);
 const [answers, setAnswers] = useState({});
 const [status, setStatus] = useState('');
 useEffect(()=> {
  fetch(`${API}/api/questions/random?count=5`)
   .then(res => res.json())
   .then(setQuestions)
   .catch(err => console.error(err));
 }, []);
 const handleChange = (qid, value) => {
  setAnswers(prev => ({ ...prev, [qid]: value }));
 };
 const handleSubmit = async (e) => {
  e.preventDefault();
  // Build answers array
  const payload = {
   userId: null,
   answers: questions.map(q => ({
    questionId: q._id,
    answer: answers[q._id] ?? null
   }))
  };
  setStatus('Submitting...');
  try {
   const res = await fetch(`${API}/api/responses`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify(payload)
    });
    const data = await res.json();
    if (data.ok) {
      setStatus('Thanks — your responses were saved!');
      setQuestions([]); // or fetch new set
    } else {
      setStatus('Error saving responses');
    }
  } catch (err) {
    console.error(err);
    setStatus('Network error');
  }
};
if (questions.length === 0) {
  return <p>Loading questions or none left...</p>;  }
return (
  <form onSubmit={handleSubmit}>
    {questions.map((q, idx) => (
      <div key={q._id} style={{ margin: '1rem 0', padding: '0.5rem', border: '1px solid #eee' }}>
        <label><strong>{idx+1}. {q.text}</strong></label>
        <div style={{ marginTop: 8 }}>
          {q.type === 'choice' ? (
            q.choices.map(choice => (
              <div key={choice}>
                <label>
                  <input
                    type="radio"
                    name={q._id}
                    value={choice}
                    checked={answers[q._id] === choice}
                    onChange={() => handleChange(q._id, choice)}
                  /> {choice}
                </label>
              </div>
            ))
          ) : (
            <textarea
              rows={3}
              style={{ width: '100%' }}
              value={answers[q._id] || ''}
              onChange={(e) => handleChange(q._id, e.target.value)}
            />
          )}
        </div>
```

```
      </div>
    ))}
    <button type="submit">Submit</button>
    <p>{status}</p>
  </form>
 ); }
```

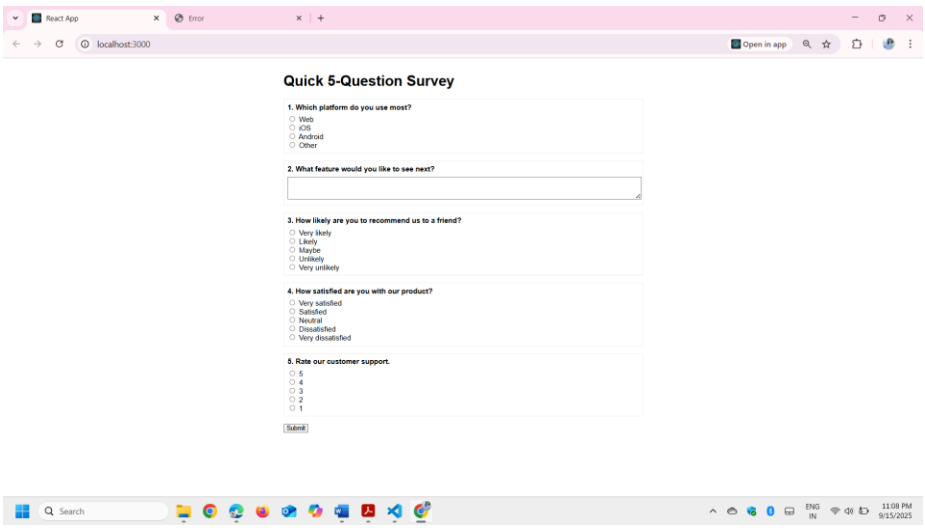
frontend/.env (optional)

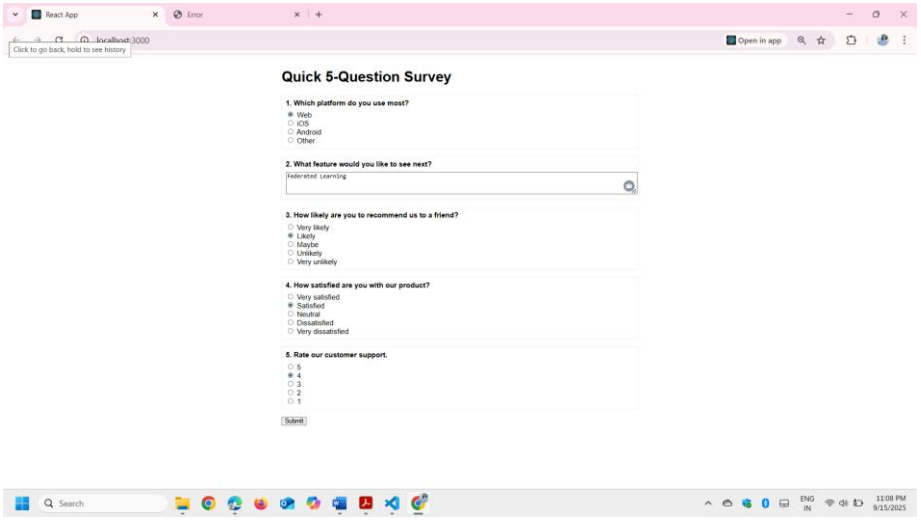
REACT_APP_API_URL=http://localhost:5000

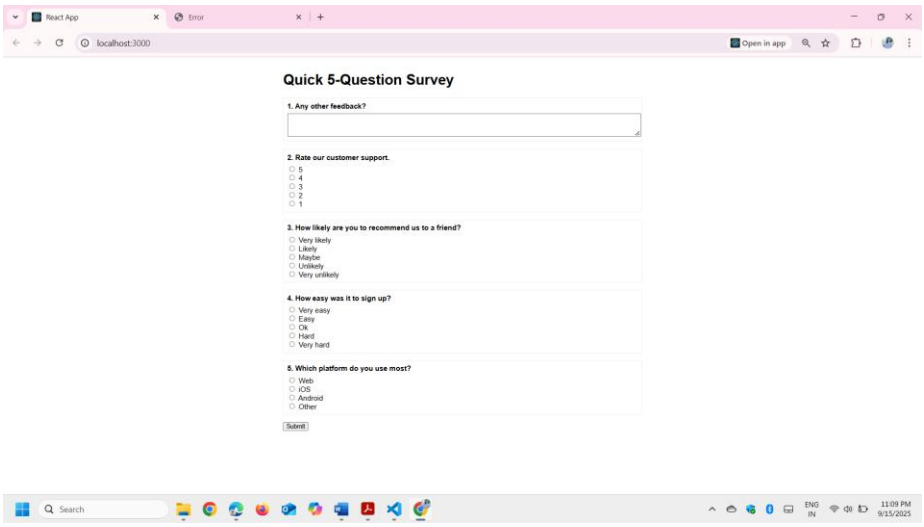
5) Run locally in VS Code — step by step

1. Open VS Code → File > Open Folder → select survey-app folder.
2. Open two terminals in VS Code (Terminal → New Terminal twice) — one for backend and one for frontend.
3. Backend:
    1. cd backend
    2. npm install
    3. Edit .env if needed (set MONGO_URI).
    4. Seed the DB: npm run seed
    5. Start server: npm run dev (uses nodemon) or npm start
    6. Server runs on http://localhost:5000 (unless you change PORT)
4. Frontend:
    1. In the other terminal: cd frontend
    2. npx create-react-app . if you didn't scaffold earlier, otherwise npm install
    3. Ensure src files match above
    4. npm start — React dev server runs on http://localhost:3000
    5. It will call the API at http://localhost:5000 (or whatever REACT_APP_API_URL you set)
5. Using one terminal (alternative): you can run both with concurrently but separate terminals are easier in VS Code.

Output







Result

The Online Survey web application was successfully developed using Node.js, Express, MongoDB, React, HTML, and CSS. Users are presented with 5 random questions drawn from a stored question collection (MCQ and text types), can answer them in a clean responsive UI, and submit their responses which are persistently stored in MongoDB.